



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA



CORSO DI LAUREA IN INFORMATICA

Tecnologie Software per il Web

AUTENTICAZIONE, RESTRIZIONE DEGLI ACCESSI E CENNI DI SICUREZZA

PROF. ATTILIO DELLA GRECA

a.a. 2025-2026

Introduzione

I due aspetti di sicurezza principali nelle applicazioni web sono:

- **Impedire agli utenti non autorizzati di accedere a dati sensibili**

Questo processo prevede la **restrizione degli accesso** (identificando quali risorse necessitano di protezione e chi dovrebbe accedervi) e l'**autenticazione** (identificando gli utenti per determinare se sono uno di quelli autorizzati)

- **Impedire agli attaccanti di rubare dati di rete mentre questi sono in transito**

Questo processo prevede l'uso di TLS (Transport Layer Security) o SSL (Secure Sockets Layer) per crittografare il traffico tra il browser e il server

- HTTPS: HTTP su una connessione crittografata SSL/TLS
- SSL è uno standard superato da TLS

I due aspetti della sicurezza web

Nelle applicazioni web la sicurezza si concentra su due obiettivi distinti ma complementari.



Controllo degli accessi

Impedire agli utenti non autorizzati di accedere ai dati sensibili

- **Restrizione degli accessi**

Identificare quali risorse vanno protette e chi può accedervi

- **Autenticazione**

Identificare l'utente per stabilire se è tra quelli autorizzati



Protezione dei dati in transito

Impedire agli attaccanti di intercettare i dati mentre viaggiano in rete

- **Cifratura del traffico**

Uso di TLS / SSL per cifrare il canale tra browser e server

- **HTTPS**

HTTP su una connessione cifrata SSL/TLS — confidenzialità e integrità

Identificazione, autenticazione, autorizzazione

Tre concetti spesso confusi: vengono applicati in sequenza, in quest'ordine preciso.



Identificazione 1

« *Chi sei?* »

L'utente dichiara la propria identità, ad esempio digitando uno username.



Autenticazione 2

« *Sei davvero tu?* »

Si verifica che l'utente sia chi dichiara di essere, ad esempio tramite la password.



Autorizzazione 3

« *Cosa puoi fare?* »

Si verifica quali risorse l'utente autenticato ha il permesso di utilizzare.

Da ricordare: l'autenticazione precede sempre l'autorizzazione. Nelle applicazioni web si parla spesso, in modo generico, di « *restrizione degli accessi* » comprendendo entrambe le fasi.

HTTPS, SSL e TLS



HTTP

Traffico in chiaro: chiunque sulla rete può leggere o alterare i dati.



HTTPS

Traffico cifrato con SSL/TLS: i dati sono protetti tra browser e server.

HTTPS

È semplicemente HTTP trasportato su una connessione cifrata SSL/TLS. Garantisce confidenzialità e integrità dei dati scambiati.

SSL

Secure Sockets Layer

Protocollo storico per cifrare le connessioni. Oggi è considerato superato e deprecato.

TLS

Transport Layer Security

Evoluzione di SSL: è lo standard attuale per cifrare il traffico di rete.

Indipendenza dei due aspetti

Gli approcci per la restrizione degli accessi sono gli stessi sia che si usi TLS/SSL sia che non lo si usi: cifratura del canale e controllo degli accessi sono problemi separati.

Restrizione degli accessi

Gli approcci per le restrizione degli accessi sono gli stessi indipendentemente dal fatto che si utilizzi o meno TLS/SSL

Essi sono:

- **Restrizione degli accessi in maniera dichiarativa**
- **Restrizione degli accessi in maniera programmatica**

Restrizione degli accessi in maniera dichiarativa

Con la restrizione degli accessi in maniera dichiarativa nessuna delle singole Servlet o pagine JSP necessita di alcun codice sensibile alla sicurezza

La restrizione degli accessi è gestita dal Servlet Container (occorre dichiarare dei parametri di configurazione in **web.xml**)

Restrizione degli accessi in maniera dichiarativa

La sicurezza come configurazione

1

Nessun codice nelle risorse

Nessuna delle singole Servlet o pagine JSP necessita di codice sensibile alla sicurezza.

2

Gestita dal Servlet Container

È il container a occuparsi della restrizione degli accessi, non lo sviluppatore.

3

Configurata in web.xml

Si dichiarano i parametri di configurazione nel descrittore di deployment web.xml.

4

Intercetta prima della risorsa

Il container applica le regole alla richiesta prima che raggiunga Servlet o JSP.

web.xml

```
<security-constraint>
  <web-resource-collection>
    <url-pattern>/admin/*</url-pattern>
  </web-resource-collection>
  <auth-constraint>
    <role-name>admin</role-name>
  </auth-constraint>
</security-constraint>
```

Come agisce il container

Richiesta

→ Container

→ Risorsa

Il controllo avviene prima che la richiesta raggiunga la risorsa.



Restrizione degli accessi in maniera programmatica

Con la sicurezza programmatica, le Servlet e le pagine JSP garantiscono la restrizione degli accessi:

Per impedire l'accesso non autorizzato, ogni Servlet o pagina JSP protetta deve verificare che l'utente sia autenticato e che abbia il permesso per accedere alla Servlet o pagina JSP richiesta

Restrizione degli accessi in maniera programmatica

La sicurezza scritta nel codice

1

Le risorse garantiscono la restrizione

Sono le Servlet e le pagine JSP stesse a implementare la sicurezza.

2

Verifica dell'autenticazione

Ogni risorsa protetta controlla che l'utente sia effettivamente autenticato.

3

Verifica dei permessi

Si controlla che l'utente abbia il diritto di accedere a quella specifica risorsa.

4

Gestione del fallimento

Se il controllo non passa, l'utente è rimandato al login o a una pagina di errore.

protected.jsp

```
// in cima a ogni risorsa
protetta
if (utenteNonAutenticato) {
    redirect("login.jsp");
    return;
}
if (permessoMancante) {
    redirect("errore.jsp");
}
```

Restrizione degli accessi: maniera dichiarativa vs. maniera programmatica

Maniera dichiarativa

- **Vantaggio:** le Servlet o pagine JSP protette non necessitano di alcun codice sensibile alla sicurezza
- **Svantaggio:** dipendente dal Servlet container, ne consegue che le applicazioni Web non sono portabili

Maniera programmatica

- **Vantaggio:** le applicazioni Web sono portabili, l'approccio è flessibile
- **Svantaggio:** il programmatore deve scrivere del codice per proteggere le Servlet o le pagine JSP che necessitano di protezione

Restrizione degli accessi: maniera dichiarativa vs. maniera programmatica

Aspetto	Maniera dichiarativa	Maniera programmatica
Dove si configura	Descrittore di deployment web.xml	Codice di Servlet e pagine JSP
Codice di sicurezza nelle risorse	Non necessario — le risorse restano pulite	Necessario — scritto dal programmatore
Gestita da	Servlet Container	Le singole Servlet e JSP
Portabilità	Bassa — dipende dal Servlet Container	Alta — l'applicazione è portabile
Flessibilità	Limitata	Elevata
In sintesi	+ Risorse senza codice di sicurezza – Applicazione non portabile	+ Portabile e flessibile – Richiede scrivere codice dedicato

Restrizione degli accessi: linee guida

01

Identificare le risorse da proteggere

Stabilire quali Servlet e quali pagine JSP necessitano effettivamente di protezione.

02

Identificare chi può accedere

Definire chi può accedere a ogni gruppo di risorse: tipicamente esistono risorse riservate al solo amministratore e risorse aperte sia all'utente sia all'amministratore.

03

Risorse riservate all'amministratore

Le pagine JSP riservate all'admin vanno nella cartella "admin"; per le Servlet l'URL pattern ha la forma /admin/myServlet.

04

Risorse comuni a utente e amministratore

Le pagine JSP accessibili a entrambi vanno nella cartella "common"; per le Servlet l'URL pattern ha la forma /common/myServlet.

Organizzazione consigliata

► myWebApp/

- login.jsp

► admin/

- protected.jsp

► common/

- protected.jsp

- Logout

In genere l'amministratore dispone delle stesse funzionalità dell'utente, più altre a lui dedicate.

Attenzione agli URL relativi

Spostare le risorse in sottocartelle cambia il modo in cui vengono risolti gli URL relativi.

Problema

Se una pagina JSP si trova nella cartella "admin", tutti gli URL relativi al suo interno fanno riferimento a quella cartella. Lo stesso vale per le Servlet.

Esempio — in `admin/protected.jsp`:

```
response.sendRedirect("login.jsp");
```

URL atteso: `myWebApp/login.jsp`

URL effettivo: `myWebApp/admin/login.jsp`

→ *il redirect punta alla cartella sbagliata!*

Soluzioni

1. URL assoluto rispetto al context path

```
response.sendRedirect(request.getContextPath() + "/login.jsp");
```

`getContextPath()` restituisce il percorso di base dell'applicazione.

2. Percorso relativo che risale di una cartella

```
response.sendRedirect("../login.jsp");
```

Il prefisso `"../"` risale alla cartella superiore prima di risolvere il percorso.

Implementazione delle restrizione degli accessi in maniera programmatica

Idea: token nella sessione

- I nomi utente e le password sono conservati in una tabella “utenti” della base di dati
- L'applicazione Web presenta una pagina di login attraverso cui l'utente fornisce il proprio nome utente e password
- Se il nome utente e la password fornita trovano riscontro nella base di dati, allora salvo nella sessione il “token”
- In ogni Servlet o pagina JSP protetta, devo assicurarmi che l'utente sia autenticato (ovvero il “token” è nella sessione) e che abbia il diritto di accedere a quella Servlet o pagina JSP
 - Se non ne ha il diritto, lo rimandiamo alla pagina di login (o ad una pagina di errore)
- Quando eseguo il logout invalido la session con **session.invalidate()**

token nella sessione



In ogni Servlet o pagina JSP protetta occorre assicurarsi che il token sia presente nella sessione e che l'utente abbia il diritto di accedere a quella risorsa.

Esempio: Token nella Session (login.jsp)

```
...
<body>
<%
List<String> errors = (List<String>) request.getAttribute("errors");
if (errors != null){
    for (String error: errors){ %>
        <%=error %> <br>
    }
}
%>
<form action="Login" method="post">
<fieldset>
    <legend>Login Custom</legend>
    <label for="username">Username</label>
    <input id="username" type="text" name="username" placeholder="enter username">
    <br>
    <label for="password">Password</label>
    <input id="password" type="password" name="password" placeholder="enter password">
    <br>
    <input type="submit" value="Login"/>
    <input type="reset" value="Reset"/>
</fieldset>
</form>
</body>
</html>
```

La pagina legge l'attributo "errors" dalla request e, se presente, stampa tutti i messaggi inviati dalla Servlet.

Esempio: Token nella Session (login.jsp)

```
...  
<body>  
<%  
List<String> errors = (List<String>) request.getAttribute("errors");  
if (errors != null){  
    for (String error: errors){ %>  
        <%=error %> <br>  
    }  
}  
<%>  
<form action="Login" method="post">  
<fieldset>  
    <legend>Login Custom</legend>  
    <label for="username">Username</label>  
    <input id="username" type="text" name="username" placeholder="enter username">  
    <br>  
    <label for="password">Password</label>  
    <input id="password" type="password" name="password" placeholder="enter password">  
    <br>  
    <input type="submit" value="Login"/>  
    <input type="reset" value="Reset"/>  
</fieldset>  
</form>  
</body>  
</html>
```

Il form invia username e password in POST alla Servlet mappata su "Login".

Esempio: Token nella Session (login.jsp)

```
...
<body>
<%
List<String> errors = (List<String>) request.getAttribute("errors");
if (errors != null){
    for (String error: errors){ %>
        <%=error %> <br>
    }
}
%>
<form action="Login" method="post">
<fieldset>
    <legend>Login Custom</legend>
    <label for="username">Username</label>
    <input id="username" type="text" name="username" placeholder="enter username">
    <br>
    <label for="password">Password</label>
    <input id="password" type="password" name="password" placeholder="enter password">
    <br>
    <input type="submit" value="Login"/>
    <input type="reset" value="Reset"/>
</fieldset>
</form>
</body>
</html>
```

La login mostra gli errori ma non li genera:
la validazione vera avviene nella Servlet.

Esempio: Token nella Session (Login.java)

```
@WebServlet("/Login")
public class Login extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");
        List<String> errors = new ArrayList<>();
        RequestDispatcher dispatcherToLoginPage = request.getRequestDispatcher("login.jsp");

        if(username == null || username.trim().isEmpty()) {
            errors.add("Il campo username non può essere vuoto!");
        }
        if(password == null || password.trim().isEmpty()) {
            errors.add("Il campo password non può essere vuoto!");
        }
        if (!errors.isEmpty()) {
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
            return; // note the return statement here!!!
        }

        username = username.trim();
        password = password.trim();

        if(username.equals("admin") && password.equals("mypass")){ //admin
            request.getSession().setAttribute("isAdmin", Boolean.TRUE); //inserisco il token nella sessione
            response.sendRedirect("admin/protected.jsp");
        } else if (username.equals("user") && password.equals("mypass")){ //user
            request.getSession().setAttribute("isAdmin", Boolean.FALSE); //inserisco il token nella sessione
            response.sendRedirect("common/protected.jsp");
        } else {
            errors.add("Username o password non validi!");
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
        }
    }
}
```

Validazione

Se i campi sono vuoti si fa forward alla login con i messaggi di errore.

Protection.zip

Esempio: Token nella Session (Login.java)

```
@WebServlet("/Login")
public class Login extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");
        List<String> errors = new ArrayList<>();
        RequestDispatcher dispatcherToLoginPage = request.getRequestDispatcher("login.jsp");

        if(username == null || username.trim().isEmpty()) {
            errors.add("Il campo username non può essere vuoto!");
        }
        if(password == null || password.trim().isEmpty()) {
            errors.add("Il campo password non può essere vuoto!");
        }
        if (!errors.isEmpty()) {
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
            return; // note the return statement here!!!
        }

        username = username.trim();
        password = password.trim();

        if(username.equals("admin") && password.equals("mypass")){ //admin
            request.getSession().setAttribute("isAdmin", Boolean.TRUE); //inserisco il token nella sessione
            response.sendRedirect("admin/protected.jsp");
        } else if (username.equals("user") && password.equals("mypass")){ //user
            request.getSession().setAttribute("isAdmin", Boolean.FALSE); //inserisco il token nella sessione
            response.sendRedirect("common/protected.jsp");
        } else {
            errors.add("Username o password non validi!");
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
        }
    }
}
```

return obbligatorio

Dopo il forward serve return per fermare l'esecuzione della Servlet.

Protection.zip

Esempio: Token nella Session (Login.java)

```
@WebServlet("/Login")
public class Login extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");
        List<String> errors = new ArrayList<>();
        RequestDispatcher dispatcherToLoginPage = request.getRequestDispatcher("login.jsp");

        if(username == null || username.trim().isEmpty()) {
            errors.add("Il campo username non può essere vuoto!");
        }
        if(password == null || password.trim().isEmpty()) {
            errors.add("Il campo password non può essere vuoto!");
        }
        if (!errors.isEmpty()) {
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
            return; // note the return statement here!!!
        }

        username = username.trim();
        password = password.trim();

        if(username.equals("admin") && password.equals("mypass")){ //admin
            request.getSession().setAttribute("isAdmin", Boolean.TRUE); //inserisco il token nella sessione
            response.sendRedirect("admin/protected.jsp");
        } else if (username.equals("user") && password.equals("mypass")){ //user
            request.getSession().setAttribute("isAdmin", Boolean.FALSE); //inserisco il token nella sessione
            response.sendRedirect("common/protected.jsp");
        } else {
            errors.add("Username o password non validi!");
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
        }
    }
}
```

Salvataggio del token

Con credenziali valide si salva isAdmin in sessione: TRUE o FALSE.

Protection.zip

Esempio: Token nella Session (Login.java)

```
@WebServlet("/Login")
public class Login extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");
        List<String> errors = new ArrayList<>();
        RequestDispatcher dispatcherToLoginPage = request.getRequestDispatcher("login.jsp");

        if(username == null || username.trim().isEmpty()) {
            errors.add("Il campo username non può essere vuoto!");
        }
        if(password == null || password.trim().isEmpty()) {
            errors.add("Il campo password non può essere vuoto!");
        }
        if (!errors.isEmpty()) {
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
            return; // note the return statement here!!!
        }

        username = username.trim();
        password = password.trim();

        if(username.equals("admin") && password.equals("mypass")){ //admin
            request.getSession().setAttribute("isAdmin", Boolean.TRUE); //inserisco il token nella sessione
            response.sendRedirect("admin/protected.jsp");
        } else if (username.equals("user") && password.equals("mypass")){ //user
            request.getSession().setAttribute("isAdmin", Boolean.FALSE); //inserisco il token nella sessione
            response.sendRedirect("common/protected.jsp");
        } else {
            errors.add("Username o password non validi!");
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
        }
    }
}
```

Redirect per ruolo

L'admin va in admin/protected.jsp, l'utente in common/protected.jsp.

Protection.zip

Esempio: Token nella Session (Login.java)

```
@WebServlet("/Login")
public class Login extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");
        List<String> errors = new ArrayList<>();
        RequestDispatcher dispatcherToLoginPage = request.getRequestDispatcher("login.jsp");

        if(username == null || username.trim().isEmpty()) {
            errors.add("Il campo username non può essere vuoto!");
        }
        if(password == null || password.trim().isEmpty()) {
            errors.add("Il campo password non può essere vuoto!");
        }
        if (!errors.isEmpty()) {
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
            return; // note the return statement here!!!
        }

        username = username.trim();
        password = password.trim();

        if(username.equals("admin") && password.equals("mypass")){ //admin
            request.getSession().setAttribute("isAdmin", Boolean.TRUE); //inserisco il token nella sessione
            response.sendRedirect("admin/protected.jsp");
        } else if (username.equals("user") && password.equals("mypass")){ //user
            request.getSession().setAttribute("isAdmin", Boolean.FALSE); //inserisco il token nella sessione
            response.sendRedirect("common/protected.jsp");
        } else {
            errors.add("Username o password non validi!");
            request.setAttribute("errors", errors);
            dispatcherToLoginPage.forward(request, response);
        }
    }
}
```

In realtà qui si dovrebbero caricare le credenziali dal DB

Esempio: Token nella Session (admin/protected.jsp)

```
...
</head>
<%
// Check user credentials
Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");
if ((isAdmin == null) || (!isAdmin)){
    response.sendRedirect(request.getContextPath() + "/login.jsp");
    return;
}
%>
<body>
<h1>Benvenuto nella Pagina Riservata all'Amministratore</h1>

<p>
Congratulazioni! <br> Questa pagina è accessibile all'amministratore soltanto (non all'utente).
</p>

<p>
<a href="<%=request.getContextPath()%>/common/Logout">Logout</a>
</p>

</body>
</html>
```

Recupero del token. La pagina legge l'attributo isAdmin dalla sessione corrente.

Esempio: Token nella Session (admin/protected.jsp)

```
...
</head>
<%
// Check user credentials
Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");
if ((isAdmin == null) || (!isAdmin)){
    response.sendRedirect(request.getContextPath() + "/login.jsp");
    return;
}
%>
<body>
<h1>Benvenuto nella Pagina Riservata all'Amministratore</h1>

<p>
Congratulazioni! <br> Questa pagina è accessibile all'amministratore soltanto (non all'utente).
</p>

<p>
<a href="<%=request.getContextPath()%>/common/Logout">Logout</a>
</p>

</body>
</html>
```

Doppia condizione. L'accesso è negato se il token è assente (utente non autenticato) oppure se isAdmin vale false (utente normale).

Esempio: Token nella Session (admin/protected.jsp)

```
...
</head>
<%
// Check user credentials
Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");
if ((isAdmin == null) || (!isAdmin)){
    response.sendRedirect(request.getContextPath() + "/login.jsp");
    return;
}
%>
<body>
<h1>Benvenuto nella Pagina Riservata all'Amministratore</h1>

<p>
Congratulazioni! <br> Questa pagina è accessibile all'amministratore soltanto (non all'utente).
</p>

<p>
<a href="<%=request.getContextPath()%>/common/Logout">Logout</a>
</p>

</body>
</html>
```

Redirect e return. Se il controllo fallisce si reindirizza alla login e si esegue return per non generare il resto della pagina.

Esempio: Token nella Session (common/protected.jsp)

```
...  
</head>  
<%  
// Check user credentials  
Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");  
if (isAdmin == null){  
    response.sendRedirect(request.getContextPath() + "/login.jsp");  
    return;  
}  
>%  
<body>  
<h1>Benvenuto nella Pagina Riservata</h1>  
  
<p>  
Congratulazioni! <br> Questa pagina è accesibile sia all'utente che all'amministratore.  
</p>  
  
<p>  
<a href="Logout">Logout</a>  
</p>  
  
</body>  
</html>
```

Qui si verifica solo `isAdmin == null`: è sufficiente essere autenticati.

Esempio: Token nella Session (common/protected.jsp)

```
...  
</head>  
<%  
// Check user credentials  
Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");  
if (isAdmin == null){  
    response.sendRedirect(request.getContextPath() + "/login.jsp");  
    return;  
}  
%>  
<body>  
<h1>Benvenuto nella Pagina Riservata</h1>  
  
<p>  
Congratulazioni! <br> Questa pagina è accesibile sia all'utente che all'amministratore.  
</p>  
  
<p>  
<a href="Logout">Logout</a>  
</p>  
  
</body>  
</html>
```

Rispetto alla pagina admin non si verifica il valore del token, ma solo la sua presenza in sessione.

La struttura del controllo resta identica: leggere il token, decidere, eventualmente reindirizzare.

Esempio: Token nella Session (Logout.java)

```
@WebServlet("/common/Logout")
public class Logout extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        Boolean isAdmin = (Boolean) request.getSession().getAttribute("isAdmin");
        if (isAdmin == null){
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        request.getSession().invalidate();
        response.sendRedirect(request.getContextPath() + "/login.jsp");
    }
}
```

Si controlla che esista una sessione autenticata: senza token non c'è nulla da invalidare.

Protection.zip

Esempio: Token nella Session (Logout.java)

```
@WebServlet("/common/Logout")
public class Logout extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        Boolean isAdmin = (Boolean) request.getSession().getAttribute("isAdmin");
        if (isAdmin == null){
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        request.getSession().invalidate();
        response.sendRedirect(request.getContextPath() + "/login.jsp");
    }
}
```

session.invalidate() distrugge la sessione e tutti i suoi attributi, token compreso.

Protection.zip

Esempio: Token nella Session (Logout.java)

```
@WebServlet("/common/Logout")
public class Logout extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        Boolean isAdmin = (Boolean) request.getSession().getAttribute("isAdmin");
        if (isAdmin == null){
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        request.getSession().invalidate();
        response.sendRedirect(request.getContextPath() + "/login.jsp");
    }
}
```

Dopo il logout l'utente viene riportato alla pagina di login.

Protection.zip

Token nella session: verso una soluzione migliore



Il problema della soluzione vista

Lo stesso blocco di controllo (token presente? diritto di accesso?) va ripetuto in ogni Servlet o pagina JSP protetta: codice duplicato, difficile da mantenere e da modificare.



La soluzione: un Filter

Un filtro è un componente che intercetta le richieste prima che raggiungano la risorsa. Il controllo di accesso viene scritto una sola volta, in un unico punto.

Controllo centralizzato

Tutta la logica di accesso in un solo componente

Zero duplicazione

Le risorse protette restano pulite, senza codice di sicurezza

Facile manutenzione

Modificare le regole significa modificare un solo file

Richiesta



Filtro di accesso



Autorizzato → risorsa

Non autorizzato → login

Il filtro decide se lasciar proseguire la richiesta o reindirizzare l'utente.

Esempio Filtro accesso (AccessControlFilter.java)

```
@WebFilter(filterName = "/AccessControlFilter", urlPatterns = "/*")
public class AccessControlFilter extends HttpFilter implements Filter {

    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpRequest = (HttpServletRequest) request;
        HttpServletResponse httpResponse = (HttpServletResponse) response;

        Boolean isAdmin = (Boolean) httpRequest.getSession().getAttribute("isAdmin");
        String path = httpRequest.getServletPath();
        System.out.println(path);
        if (path.contains("/common/") && isAdmin==null) {
            httpResponse.sendRedirect(httpRequest.getContextPath() + "/login.jsp");
            return;
        } else if (path.contains("/admin/") && (isAdmin==null || !isAdmin)) {
            httpResponse.sendRedirect(httpRequest.getContextPath() + "/login.jsp");
            return;
        }

        chain.doFilter(request, response);
    }
}
```

Intercetta tutto. urlPatterns = "/*"
fa sì che il filtro intercetti ogni
richiesta in arrivo.

ProtectionFilters.zip

Esempio Filtro accesso (AccessControlFilter.java)

```
@WebFilter(filterName = "/AccessControlFilter", urlPatterns = "/*")
public class AccessControlFilter extends HttpFilter implements Filter {

    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpServletRequest = (HttpServletRequest) request;
        HttpServletResponse httpServletResponse = (HttpServletResponse) response;

        Boolean isAdmin = (Boolean) httpServletRequest.getSession().getAttribute("isAdmin");
        String path = httpServletRequest.getServletPath();
        System.out.println(path);
        if (path.contains("/common/") && isAdmin==null) {
            httpServletResponse.sendRedirect(httpServletRequest.getContextPath() + "/login.jsp");
            return;
        } else if (path.contains("/admin/") && (isAdmin==null || !isAdmin)) {
            httpServletResponse.sendRedirect(httpServletRequest.getContextPath() + "/login.jsp");
            return;
        }

        chain.doFilter(request, response);
    }
}
```

Risorse in /common/. È sufficiente essere autenticati: si verifica solo che il token non sia null.

ProtectionFilters.zip

Esempio Filtro accesso (AccessControlFilter.java)

```
@WebFilter(filterName = "/AccessControlFilter", urlPatterns = "/*")
public class AccessControlFilter extends HttpFilter implements Filter {

    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpRequest = (HttpServletRequest) request;
        HttpServletResponse httpResponse = (HttpServletResponse) response;

        Boolean isAdmin = (Boolean) httpRequest.getSession().getAttribute("isAdmin");
        String path = httpRequest.getServletPath();
        System.out.println(path);
        if (path.contains("/common/") && isAdmin==null) {
            httpResponse.sendRedirect(httpRequest.getContextPath() + "/login.jsp");
            return;
        } else if (path.contains("/admin/") && (isAdmin==null || !isAdmin)) {
            httpResponse.sendRedirect(httpRequest.getContextPath() + "/login.jsp");
            return;
        }

        chain.doFilter(request, response);
    }
}
```

Risorse in /admin/. Serve essere autenticati e avere isAdmin true: l'utente normale viene escluso.

ProtectionFilters.zip

Esempio Filtro accesso (AccessControlFilter.java)

```
@WebFilter(filterName = "/AccessControlFilter", urlPatterns = "/*")
public class AccessControlFilter extends HttpFilter implements Filter {

    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpRequest = (HttpServletRequest) request;
        HttpServletResponse httpResponse = (HttpServletResponse) response;

        Boolean isAdmin = (Boolean) httpRequest.getSession().getAttribute("isAdmin");
        String path = httpRequest.getServletPath();
        System.out.println(path);
        if (path.contains("/common/") && isAdmin==null) {
            httpResponse.sendRedirect(httpRequest.getContextPath() + "/login.jsp");
            return;
        } else if (path.contains("/admin/") && (isAdmin==null || !isAdmin)) {
            httpResponse.sendRedirect(httpRequest.getContextPath() + "/login.jsp");
            return;
        }

        chain.doFilter(request, response);
    }
}
```

chain.doFilter(). Se i controlli passano, lascia proseguire la richiesta verso la risorsa richiesta.

ProtectionFilters.zip

Esempio Filtro accesso (common/protected.jsp)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Pagina Riservata</title>
</head>

<body>
<h1>Benvenuto nella Pagina Riservata</h1>

<p>
Congratulazioni! <br> Questa pagina è accesibile sia all'utente che all'amministratore.
</p>

<p>
<a href="Logout">Logout</a>
</p>

</body>
</html>
```

Le pagine JSP protette non contengono più alcun codice di sicurezza: tutto è nel filtro.

ProtectionFilters.zip

Creare il certificato SSL/TLS in tomcat

01

Generare il certificato con Keytool

Si usa lo strumento Keytool incluso in Java, eseguendolo dal prompt dei comandi.

02

Seguire le istruzioni a schermo

Keytool chiede alcune informazioni; al termine viene creato il file keystore.jks nella cartella corrente.

03

Spostare il file keystore.jks

In produzione: nella directory principale di Tomcat. In sviluppo: nella cartella temporanea del server in Eclipse.

Prompt dei comandi

```
keytool -genkey -alias tomcat  
-keypass myPassword  
-keystore keystore.jks  
-storepass myPassword  
-keyalg RSA -validity 360  
-keysize 2048
```

Un certificato autogenerato serve solo per test e sviluppo: il browser lo segnala come non attendibile. In un ambiente reale il certificato SSL/TLS viene rilasciato da una Certification Authority (CA) riconosciuta.

Configurare il connettore — server.xml

server.xml

```
<!-- Connettore HTTPS sulla porta 443 -->
<Connector port="443"
  protocol="...Http11NioProtocol"
  maxThreads="150" SSLEnabled="true"
  maxParameterCount="1000">
  <UpgradeProtocol
    className="...http2.Http2Protocol" />
  <SSLHostConfig>
    <Certificate
      certificateKeystoreFile="keystore.jks"
      certificateKeystorePassword="myPassword"
      type="RSA"/>
    </SSLHostConfig>
  </Connector>

<!-- Redirect del connettore HTTP -->
<Connector connectionTimeout="20000"
  port="80" protocol="HTTP/1.1"
  redirectPort="443"/>
```

Come funziona

1. **Connettore HTTPS.** Il connettore sulla porta 443 abilita SSL/TLS e punta al keystore.jks creato in precedenza.
2. **redirectPort.** Nel connettore HTTP (porta 80) la redirectPort va impostata a 443, così le richieste in chiaro vengono dirottate su HTTPS.
3. **Avvio.** Dopo aver salvato server.xml si avvia Tomcat per applicare la configurazione.
4. **Accesso.** L'applicazione si raggiunge con `https://localhost/yourWebApp/...`

Impedire il furto dei dati in transito

Una volta configurato Tomcat per SSL/TLS, occorre garantire che le risorse vengano usate via HTTPS.



Maniera programmatica

- Ogni Servlet o JSP verifica il protocollo usato con `request.isSecure()`
- Se la richiesta arriva in HTTP semplice, viene reindirizzata manualmente all'equivalente HTTPS
- Anche qui si possono usare i filtri per centralizzare il controllo in un unico punto



Maniera dichiarativa

- Si aggiungono parametri di configurazione al descrittore `web.xml`
- Si definisce un vincolo di sicurezza che impone il canale cifrato
- Il container reindirizza automaticamente le richieste in chiaro su HTTPS

In entrambi i casi Tomcat deve essere stato configurato per SSL/TLS (Slide precedenti).

Esempio dichiarativo security-constraint

web.xml

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>
      All
    </web-resource-name>
    <url-pattern>/*</url-pattern>
  </web-resource-collection>
  <user-data-constraint>
    <transport-guarantee>
      CONFIDENTIAL
    </transport-guarantee>
  </user-data-constraint>
</security-constraint>
```

Come funziona

- **Dove si scrive.** Il vincolo si dichiara nel descrittore di deployment web.xml dell'applicazione.
- **url-pattern /*.** Il pattern /* applica il vincolo a tutte le risorse della web application.
- **transport-guarantee CONFIDENTIAL.** Impone che la risorsa sia raggiunta solo tramite un canale cifrato, cioè HTTPS.
- **Effetto automatico.** Ogni tentativo di accesso in HTTP semplice viene reindirizzato dal container su HTTPS.

Nessun codice nelle Servlet o JSP: la protezione del canale è pura configurazione.

Attacchi di tipo Clickjacking



Che cos'è

Una tecnica di attacco che inganna l'utente inducendolo a cliccare su qualcosa di diverso da ciò che percepisce, sfruttando elementi nascosti o trasparenti sovrapposti a contenuti visibili.

Come funziona



Pagina trappola

L'attaccante crea una pagina web che incorpora, ad esempio tramite un `<iframe>`, un altro sito legittimo.



Elemento invisibile

Sopra il pulsante reale (« Mi piace », « Acquista », « Autorizza »...) viene posizionato un elemento invisibile o trasparente.



Click ingannato

L'utente crede di cliccare un bottone innocuo, ma in realtà interagisce col contenuto nascosto, compiendo azioni a sua insaputa.

Difendersi dal Clickjacking — web.xml

web.xml

```
<filter>
  <filter-name>httpHeaderSecurity</filter-name>
  <filter-class>
    org.apache.catalina.filters
      .HttpHeaderSecurityFilter
  </filter-class>
  <init-param>
    <param-name>
      antiClickJackingEnabled</param-name>
    <param-value>true</param-value>
  </init-param>
  <init-param>
    <param-name>
      antiClickJackingOption</param-name>
    <param-value>DENY</param-value>
  </init-param>
</filter>
<filter-mapping>
  <filter-name>httpHeaderSecurity</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

Come funziona

- **HttpHeaderSecurityFilter.** Tomcat mette a disposizione un filtro pronto che aggiunge header HTTP di sicurezza alle risposte.
- **antiClickJackingEnabled.** Impostato a true, attiva l'aggiunta dell'header X-Frame-Options.
- **antiClickJackingOption DENY.** Impedisce che le pagine vengano caricate dentro un <iframe>, in qualsiasi contesto.
- **Risultato.** Senza la possibilità di incapsulare il sito in un iframe, l'attacco di clickjacking non può funzionare.

Si dichiara nel web.xml e si applica a tutte le risorse con il pattern /*.

Memorizzazione sicura delle password

- **Niente testo in chiaro**

Le password non devono mai essere memorizzate in chiaro nel database.

- **Si memorizza l'hash**

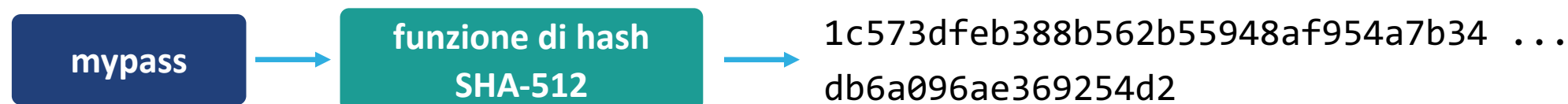
Al posto della password si salva nel database il suo hash.

- **Che cos'è un hash**

Una funzione non invertibile che mappa una stringa di lunghezza arbitraria in una stringa di lunghezza predefinita.

- **Verifica al login**

Si calcola l'hash della password fornita e lo si confronta con quello memorizzato: se coincidono, la password è corretta.



Calcolare l'hash — metodo toHash()

```
private String toHash(String password) {
    String hashString = null;
    try {
        java.security.MessageDigest digest = java.security.MessageDigest.getInstance("SHA-512");
        byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
        hashString = "";
        for (int i = 0; i < hash.length; i++) {
            hashString += Integer.toHexString(
                Come funziona (hash[i] & 0xFF) | 0x100
                ).substring(1,3);
        }
    } catch (java.security.NoSuchAlgorithmException e) {
        System.out.println(e);
    }
    return hashString;
}
```

MessageDigest SHA-512. Si ottiene un'istanza dell'algoritmo SHA-512, più sicuro di SHA-1 e SHA-256.

Esempio: l'hash della password “mypass” è:

“1c573dfefb388b562b55948af954a7b344dde1cc2099e978a992790429e7c01a4205506a
93d9aef3bab32d6f06d75b7777a7ad8859e672fedb6a096ae369254d2”

Calcolare l'hash — metodo toHash()

```
private String toHash(String password) {
    String hashString = null;
    try {
        java.security.MessageDigest digest = java.security.MessageDigest.getInstance("SHA-512");
        byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
        hashString = "";
        for (int i = 0; i < hash.length; i++) {
            hashString += Integer.toHexString(
                Come funziona (hash[i] & 0xFF) | 0x100
                ).substring(1,3);
        }
    } catch (java.security.NoSuchAlgorithmException e) {
        System.out.println(e);
    }
    return hashString;
}
```

digest(). Calcola l'hash a partire dai byte della password, codificata in UTF-8.

Esempio: l'hash della password “mypass” è:

“1c573dfeb388b562b55948af954a7b344dde1cc2099e978a992790429e7c01a4205506a
93d9aef3bab32d6f06d75b7777a7ad8859e672fedb6a096ae369254d2”

Calcolare l'hash — metodo toHash()

```
private String toHash(String password) {
    String hashString = null;
    try {
        java.security.MessageDigest digest = java.security.MessageDigest.getInstance("SHA-512");
        byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
        hashString = "";
        for (int i = 0; i < hash.length; i++) {
            hashString += Integer.toHexString(
                Come funziona (hash[i] & 0xFF) | 0x100
                ).substring(1, 3);
        }
    } catch (java.security.NoSuchAlgorithmException e) {
        System.out.println(e);
    }
    return hashString;
}
```

Conversione esadecimale. Il ciclo trasforma ogni byte in due cifre esadecimali, costruendo la stringa hash finale.

Esempio: l'hash della password "mypass" è:

"1c573dfeb388b562b55948af954a7b344dde1cc2099e978a992790429e7c01a4205506a
93d9aef3bab32d6f06d75b7777a7ad8859e672fedb6a096ae369254d2"

Calcolare l'hash — metodo toHash()

```
private String toHash(String password) {
    String hashString = null;
    try {
        java.security.MessageDigest digest = java.security.MessageDigest.getInstance("SHA-512");
        byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
        hashString = "";
        for (int i = 0; i < hash.length; i++) {
            hashString += Integer.toHexString(
                Come funziona (hash[i] & 0xFF) | 0x100
                ).substring(1,3);
        }
    } catch (java.security.NoSuchAlgorithmException e) {
        System.out.println(e);
    }
    return hashString;
}
```

Uso al login. Si confronta toHash(password inserita) con l'hash memorizzato nel database.

Esempio: l'hash della password “mypass” è:

“1c573dfefb388b562b55948af954a7b344dde1cc2099e978a992790429e7c01a4205506a
93d9aef3bab32d6f06d75b7777a7ad8859e672fedb6a096ae369254d2”